

Quantifying the Cost of Complexity: Evaluating Noise-Based vs. Bell State Quantum Random Number Generators.

Peter Jaš

Department of Computers and Informatics

Technical University of Košice

Košice, Slovakia

peterjas81@gmail.com

Abstract—Bell-pair quantum random number generators (QRNGs) are often assumed to be superior to other entropy sources, since measuring an entangled pair provides both randomness and a built-in error signal: outcomes inconsistent with the ideal Bell state can be discarded before contaminating the output. A preliminary study of mine suggested otherwise, motivating a direct test of this assumption. We implement two QRNG designs on two IBM Heron-generation processors (IBM Fez and IBM Marrakesh) using topology-aware qubit selection: a depth-1 Hadamard circuit measuring every healthy qubit independently as a noise-based baseline, and a Bell-pair circuit with maximum-matching layout that discards forbidden (non-Bell) outcomes as its error-detection mechanism. Each of the four design-processor combinations produced approximately 10 million raw bits, evaluated under an identical four-layer statistical framework (segment-wise bias, min-entropy, the full NIST SP 800-22 battery, bias-robust next-bit prediction, and multi-order Markov dependency) alongside a matched-throughput cost analysis in shots, QPU-seconds, and USD. Contrary to theoretical expectation, Bell-pair error detection did not improve output quality: relative to Hadamard, its raw streams showed roughly an order of magnitude more bias (0.0175, 0.0162 vs. 0.0018, 0.0025), lower min-entropy (0.9504, 0.9539 vs. 0.9949, 0.9927), and lower NIST pass rates (0.45, 0.48 vs. 0.62, 0.68) on both processors, while costing $\sim 2.8\times$ more shots and QPU time per usable bit. After SHA-512 whitening the two designs become statistically indistinguishable, but the cost and entropy-density gap persists. The result is a clear negative finding: on current IBM Heron hardware, Bell-pair error detection provides no measurable quality benefit at substantial extra cost, making the simpler Hadamard design the better choice.

Index Terms—quantum random number generation, Bell pairs, IBM Heron, superconducting qubits, cost-quality trade-off, NIST SP 800-22, entropy extraction, error detection

I. INTRODUCTION

Random numbers underpin cryptography, key generation, Monte Carlo simulation, and randomized algorithms. Classical PRNGs (e.g., Mersenne Twister) are deterministic: an adversary who learns the internal state can reconstruct all outputs. Quantum random number generators (QRNGs) avoid this by grounding unpredictability in the Born rule, making measurement outcomes on superposed states irreducibly non-deterministic [1], [2].

Bell-pair QRNGs are notable among these designs. Measuring the entangled state $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$ ideally yields one bit of entropy per pair, plus a built-in check: outcomes in $\{|01\rangle, |10\rangle\}$ are theoretically impossible, so their appearance signals hardware error (decoherence, gate or readout error). Discarding these *forbidden states* thus gives hardware-free error detection in principle a more rigorous choice than a single-qubit Hadamard measurement, which offers no such check.

In practice, this advantage carries a cost. Bell-pair circuits need a lower-fidelity two-qubit gate, use two qubits per bit instead of one, and require physically coupled qubit pairs. A depth-1 Hadamard circuit avoids all of this: every healthy qubit yields an independent bit each shot. The real question is not whether Bell-pair error detection is elegant, but whether it is *worth its price* on current hardware.

A preliminary study by the author [9] on IQM Emerald (54 qubits, via Amazon Braket) found that *including* rather than discarding forbidden states cut global bias by 42% (0.0233 $\rightarrow$ 0.0164). This result was limited to a 25,000-bit sample, one processor, one session, and lacked any comparison to a simpler baseline leaving open whether Bell-pair complexity is justified at all.

This paper resolves that question. We run both designs on two IBM Heron processors (Fez and Marrakesh), collecting ~ 10 M raw bits per (design, processor) pair. We evaluate each stream via segment-wise bias, min-entropy, the full NIST SP 800-22 suite, next-bit prediction, and multi-order Markov dependency, alongside a matched-throughput cost analysis in shots, QPU-seconds, and dollars.

Research Question

Does the error detection built into a Bell-pair QRNG design produce statistically better bits than a raw, low-cost, depth-1 Hadamard QRNG on current superconducting hardware, and is any quality advantage large enough to justify the associated cost premium?

Objectives

- 1) Implement a topology-aware depth-1 striped Hadamard QRNG that measures every healthy qubit in parallel.
- 2) Implement a topology-aware Bell-pair QRNG with maximum matching and forbidden-state discard.
- 3) Compare the statistical quality of both designs on two IBM Heron processors using an identical four-layer evaluation framework.
- 4) Compare implementation cost in shots, QPU-seconds, and USD.
- 5) Determine whether Bell-pair error detection is worthwhile at current hardware fidelities.

Contributions

To our knowledge this is the first large-scale ($\sim 10^7$ bits per condition), controlled, side-by-side comparison of Bell-pair versus depth-1 Hadamard QRNGs on cloud-accessible superconducting hardware. The contributions are:

- A cost-quality analysis showing that, on current IBM Heron hardware, Bell-pair error detection produces *worse* raw bit-stream quality than the Hadamard baseline while costing approximately $2.8\times$ more per usable bit.
- Validation of that conclusion across two independent Heron processors (IBM Fez, IBM Marrakesh), demonstrating that the effect is not processor-specific.
- Empirical evidence that a strong entropy extractor (SHA-512) equalises the two designs after post-processing, but at that point the throughput and price advantages of the simpler design remain decisive.
- A recontextualisation of the preliminary IQM Emerald finding: the earlier observation was about a policy choice *within* the Bell-pair design (include vs. discard) and does not challenge the present result, which compares Bell-pair *as a design* against the Hadamard baseline.

II. BACKGROUND AND PRELIMINARY STUDY

A. Quantum Randomness

Quantum randomness can be extracted from a single qubit via a Hadamard gate, or from a pair via the Bell state $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$, as described in our preliminary study [9]. Both constructions ground unpredictability in the Born rule: measurement outcomes are genuinely undetermined, not merely unknown, and each design yields one bit of entropy per measurement (Shannon for the single qubit, von Neumann for the Bell pair) [1]. Ideally, Bell-pair measurement yields only $\{|00\rangle, |11\rangle\}$.

B. Hardware Imperfections and Forbidden States

Real hardware perturbs both designs via T_1/T_2 relaxation, gate infidelity, and readout error. For Hadamard, this appears as residual bias but never destroys the source: every bit is still a bit, just imperfectly balanced. For Bell pairs, the same errors produce outcomes forbidden by the ideal state typically several percent of shots on current superconducting processors. Because these outcomes are theoretically impossible, they are identifiable without external reference, giving Bell-pair circuits

an implicit error-detection channel Hadamard circuits lack. This is the strongest argument for the Bell-pair design.

C. Preliminary Study

Our preliminary work [9] implemented a Bell-pair QRNG on IQM Emerald (54 qubits, via Amazon Braket), using maximum matching on the coupler graph to instantiate 25 independent pairs per shot; 1,000 shots yielded 25,000 candidate bits.

The key finding concerned forbidden-state handling: the *discard* policy gave a global bias of 0.0233, while an *include* policy (mapping $|01\rangle \rightarrow 1$, $|10\rangle \rightarrow 0$) reduced bias to 0.0164 and raised throughput by 12.4%. Both policies passed the frequency and cumulative-sums NIST tests, with no degradation in segment-level homogeneity preliminary evidence that hardware error states might act as a bias-cancelling source rather than pure noise.

D. Limitations of the Preliminary Study and the Present Research Gap

That study flagged three limitations the present work removes: (i) a 25,000-bit sample, well below the 10^6 bits recommended for reliable NIST SP 800-22 evaluation [3]; (ii) a single session on a single processor, leaving calibration effects unresolved; and (iii) no comparison against a simpler baseline, so it could not establish whether Bell-pair complexity is worth its cost.

The preliminary result is a statement *within* the Bell-pair design (include beats discard), not one *about* Bell-pair QRNGs relative to alternatives. If a bare Hadamard circuit matches or beats it at a fraction of the cost, the Bell-pair line is hard to justify regardless of forbidden-state policy the gap this work fills via a large-scale controlled comparison.

III. RESEARCH METHODOLOGY

A. Experimental Design

The experiment factorially crosses two QRNG designs (Hadamard, Bell-pair) with two IBM Heron-generation processors (IBM Fez, IBM Marrakesh), producing four independent bit-streams. Each stream is generated using topology-aware qubit selection under identical calibration snapshots taken from IBM Quantum’s public backend properties API on the day of collection. Each stream targets $\sim 10^7$ raw bits, which is $10\times$ the NIST SP 800-22 recommended minimum [3]. In sequence: (i) a backend calibration snapshot is fetched, (ii) healthy qubits and healthy couplers are selected, (iii) the target circuit (Hadamard or Bell-pair) is synthesised, (iv) jobs are submitted through Qiskit Runtime, (v) raw bits are extracted, (vi) SHA-512 whitening is optionally applied, and (vii) the streams are submitted to the four-layer statistical evaluation and cost accounting.

B. Hardware

Both processors belong to IBM’s Heron family (revision 2) and expose 156 physical qubits arranged on a heavy-hex-derived lattice with tunable couplers. Native gates are single-

TABLE I: IBM Heron processors used in this study.

Parameter	IBM Fez	IBM Marrakesh
Family / revision	Heron / 2	Heron / 2
Total qubits	156	156
Healthy qubits (used)	124	113
Healthy couplers	>100	>90
Bell pairs (max matching)	43	44
Hadamard bits per shot	124	113

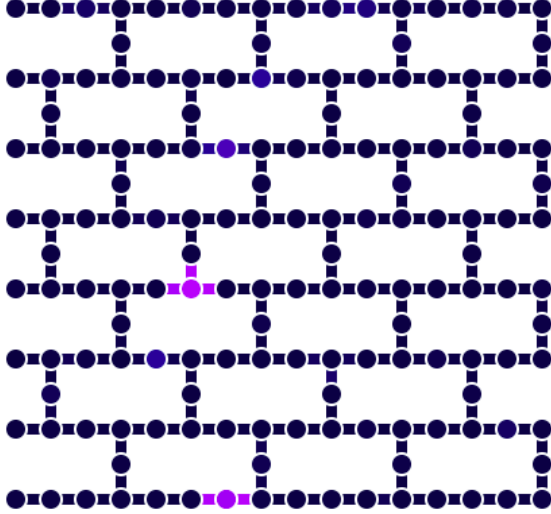


Fig. 1: IBM Heron 156-qubit heavy-hex-derived topology used in this study. Dark nodes are healthy qubits selected for both designs; lighter/violet nodes were excluded as unhealthy at the calibration snapshot in force at collection time. Edges are physical couplers; the maximum matching used for the Bell-pair layout is a subset of the healthy edges.

qubit R_Z , $\sqrt{X}$, and the two-qubit CZ or ECR (backend-dependent) entangling gate. Access was through IBM Quantum Runtime [7]. Table I summarises the two backends as observed at collection time. Fig. 1 shows the IBM Heron topology with the healthy qubits used in this study highlighted.

A qubit was retained if its readout assignment error and single-qubit gate error at collection time were both below the median value across the device; couplers were retained analogously for two-qubit gate error. Unhealthy qubits were excluded from both designs so that the comparison is not contaminated by known-bad hardware.

C. QRNG Designs

Depth-1 striped Hadamard QRNG. A single Hadamard gate is applied to every healthy qubit in parallel, followed by simultaneous measurement of every measured qubit. Each shot yields as many raw bits as there are healthy qubits: 124 on Fez, 113 on Marrakesh. The circuit is depth-1 (excluding the terminating measurement layer) and contains no two-qubit gates, so it is minimally exposed to entangling-gate error

and to the accumulated decoherence of longer schedules. The construction is given in Algorithm 2.

Bell-pair QRNG with maximum matching. The processor’s healthy-coupler graph is treated as an undirected graph $G = (V, E)$. A maximum matching $E' \subseteq E$ is computed such that no two selected edges share a vertex (as in the preliminary study [9]), and each matched edge (q_i, q_j) hosts an independent Bell-pair circuit: H on q_i followed by CNOT on (q_i, q_j) , followed by simultaneous measurement of both qubits. The matching yields 43 pairs on Fez and 44 pairs on Marrakesh. Each pair contributes one candidate bit per shot when the outcome lies in $\{|00\rangle, |11\rangle\}$; forbidden outcomes $\{|01\rangle, |10\rangle\}$ are discarded, applying the standard error detection semantics. We use the discard policy here because it is the semantics under which Bell-pair error detection can be claimed as a feature; the include policy of [9] is a separate question orthogonal to this comparison. The construction is given in Algorithm 3.

D. Data Collection

Circuits were submitted through Qiskit Runtime in batches. To reach the $\sim 10^7$ -bit target, the Hadamard design used approximately 80,000–85,000 shots per backend, while the Bell-pair design used approximately 230,000–240,000 shots per backend a direct consequence of Bell-pair circuits producing many fewer bits per shot (see Sec. V). Jobs of at most 50,000 shots were used; the QPU-seconds and USD costs reported by Runtime were summed across jobs. Whitening was performed offline using SHA-512 in a fixed 128-input-bit $\rightarrow$ 96-output-bit configuration retaining approximately 75% of the entropy of the raw stream.

E. Statistical Evaluation

Each of the eight streams (four raw + four whitened) is submitted to the same four-layer framework of [9]. The design deliberately does not rely on any single test.

- *Bias and min-entropy.* We report segment-wise bias, computed as $|p_1 - 0.5|$ over 20 equal-length partitions of each stream, together with the per-bit min-entropy $-\log_2 \max(p_0, p_1)$.
- *NIST SP 800-22.* The full 15-test battery [3] is applied to five independent 1,000,000-bit partitions of each stream; a test passes on a partition when $p > 0.01$.
- *Next-bit prediction.* A logistic-regression classifier is trained on windows of the previous W bits with $W \in \{8, 16, 24\}$, evaluated by 8-fold time-ordered cross-validation. The primary signal is ROC-AUC with a 95% Hanley–McNeil confidence interval, because AUC is bias-robust: a biased but memoryless source still gives $\text{AUC} \approx 0.5$, whereas plain accuracy rises just from majority guessing. A window passes when its AUC 95% CI includes 0.5 and accuracy does not significantly exceed the majority baseline; the stream verdict is the AND across all windows.
- *Multi-order Markov dependency.* A k -th order Markov predictor is tested for $k \in \{1, 2, 4\}$; the reported quantity

Algorithm 1 Healthy qubit selection and maximum matching

```

1: Fetch backend properties  $P$  from IBM Quantum Runtime
2:  $V \leftarrow \{q : \text{readout\_err}(q) < \text{median} \wedge \text{gate\_err}(q) < \text{median}\}$ 
3:  $E \leftarrow \{(u, v) \in \text{coupling\_map} : u, v \in V \wedge \text{cx\_err}(u, v) < \text{median}\}$ 
4: if design is Hadamard then
5:   return  $V$  {use every healthy qubit as an independent bit source}
6: else
7:    $M \leftarrow \text{MAXMATCHING}(V, E)$  {Blossom algorithm}
8:   return  $M$  {list of independent Bell-pair edges}
9: end if

```

is accuracy *above* the majority baseline, which is likewise bias-robust.

F. Cost Evaluation

Cost is measured in three commensurable units: (i) *shots* needed per 10^6 usable bits, (ii) *QPU-seconds* on the target processor as reported by Runtime, and (iii) *USD* at the pay-as-you-go rate (\$1.60/s at the time of collection). For the Bell-pair design the discard policy reduces yield by the observed forbidden-state fraction, and shots-per-bit is computed on the yielded, not the raw, bit count.

IV. IMPLEMENTATION

A. Software Stack

The generator was implemented in Python 3.11 using Qiskit 1.x and IBM Quantum Runtime [7]. All circuits were transpiled with `optimization_level=3` against the target backend’s calibrated coupling map so that no additional SWAP overhead was introduced. Backend calibration data was fetched from the Runtime service immediately before circuit synthesis, ensuring healthy-qubit selection was based on the calibration snapshot in force at execution time. Analysis was performed with NumPy, SciPy, and scikit-learn; the NIST SP 800-22 battery was run through a reference Python port.

B. Backend Selection and Maximum Matching

Algorithm 1 summarises the healthy-qubit selection and maximum-matching procedure common to both designs.

C. Circuit Construction

Algorithm 2 constructs the depth-1 striped Hadamard circuit. Because every gate is single-qubit and every qubit is measured simultaneously, the circuit’s total depth after transpilation remains constant at 1 gate plus the measurement layer, regardless of backend size. ple, but the study could not determine whether the added circuit complexity of Bell-pair QRNGs is justified at all on contemporary superconducting hardware. This work resolves that question through a large-scale, controlled comparison. We implement two QRNGs on two IBM Heron-generation processors (IBM Fez and IBM

Algorithm 2 Depth-1 striped Hadamard QRNG

```

1: Given healthy qubit set  $V$  and shot count  $N$ 
2: Initialise  $|\psi\rangle = |0\rangle^{\otimes |V|}$  on  $V$ 
3: for each  $q \in V$  in parallel do
4:   apply  $H(q)$ 
5: end for
6: measure all  $q \in V$  into classical register  $c$ 
7: run  $N$  shots; concatenate every shot’s  $|V|$ -bit outcome
8: return bit stream of length  $N \cdot |V|$ 

```

Algorithm 3 Bell-pair QRNG with maximum matching (discard policy)

```

1: Given matching  $M$  and shot count  $N$ 
2: Initialise  $|\psi\rangle = |0\rangle^{\otimes 2|M|}$ 
3: for each  $(q_i, q_j) \in M$  in parallel do
4:   apply  $H(q_i)$ 
5:   apply  $\text{CNOT}(q_i, q_j)$ 
6: end for
7: measure all qubits in  $M$ ; obtain per-pair outcomes  $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$ 
8: for each shot, each pair outcome  $o$  do
9:   if  $o \in \{|00\rangle, |11\rangle\}$  then
10:    emit bit  $f(o)$  where  $f(|00\rangle) = 0$ ,  $f(|11\rangle) = 1$ 
11:   else
12:    discard {error-detected pair}
13:   end if
14: end for
15: return concatenated bit stream

```

Marrakesh) using topology-aware qubit selection: (i) a depth-1 striped Hadamard circuit that measures every healthy qubit independently, and (ii) a Bellpair circuit with maximum-matching layout and forbidden-state discard. Each configuration produced approximately 10 million raw bits per processor, evaluated with a four-layer framework covering segment-wise bias, min-entropy, the full NIST SP 800-22 battery, bias-robust ROC-AUC next-bit prediction, and multiorder Markov dependency, together with a matched-throughput cost analysis in shots, QPU-seconds, and USD. The Hadamard QRNG yields an order of magnitude lower raw bias (0.0018 and 0.00

Algorithm 3 constructs the Bell-pair circuit. Each matched edge produces an independent Bell pair; measurements of forbidden outcomes are filtered downstream.

D. Statistical Framework and Reproducibility

The evaluation pipeline is fully deterministic: raw bit streams are stored as text files together with their generation metadata (backend name, calibration snapshot, shot count, job IDs, timestamps), and the analysis scripts operate on those files without re-invoking the QPU. This allows independent re-analysis of the same streams. Full source code and data files are available on request.

V. EXPERIMENTAL RESULTS

A. Data Collected

Table II summarises the four raw streams. Bell-pair streams are shorter than the ratio of shots would suggest because the discard policy yielded approximately 2.94% (Fez) and 3.18% (Marrakesh) forbidden-state rejections.

TABLE II: Collected data by design and backend.

Design/Backend	Shots	Bits (raw)	QPU s	Cost (USD)
Hadamard/F	85,000	10,5 mil.	26	41.60
Bell/F	240,000	10, mil.	73	116.80
Hadamard/M	80,000	9, mil.	26	41.60
Bell/M	230,000	9,8 mil.	74	118.40

B. Segment-wise Bias and Min-Entropy

Aggregated over the full stream, raw bias is an order of magnitude lower for the Hadamard design than for the Bell-pair design on both processors: 0.0018 versus 0.0175 on Fez, and 0.0025 versus 0.0162 on Marrakesh. A single whole-stream number can mask transient drift or a handful of bad segments, so Fig. 2 and Fig. 3 plot the same quantity, $|p_1 - 0.5|$, computed on 20 equal-length partitions of each stream a more informative view because it shows whether the gap is persistent or incidental. On both processors the raw Bell-pair trace sits roughly an order of magnitude above the raw Hadamard trace *throughout the run*, showing that the excess bias is a structural property of the design rather than a localised drift or a transient calibration excursion. Whitened traces (dashed) collapse to near zero regardless of the raw stream’s behaviour, confirming that the extractor removes residual imbalance uniformly.

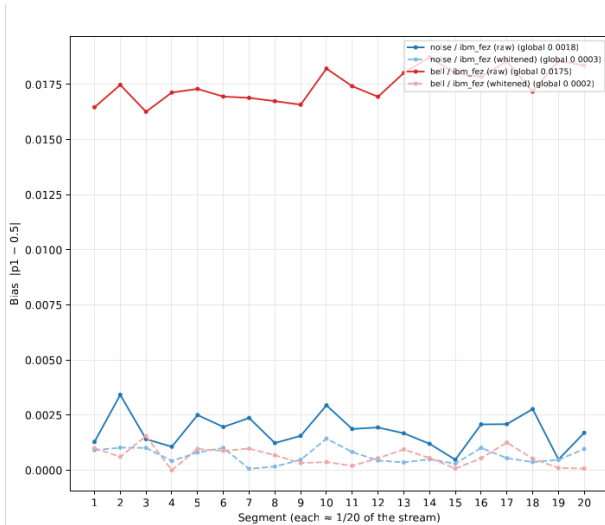


Fig. 2: Segment-wise bias $|p_1 - 0.5|$ across 20 equal partitions of each stream, IBM Fez. Solid lines are raw streams, dashed lines are whitened. The raw Bell-pair trace sits an order of magnitude above the raw Hadamard trace throughout the run.

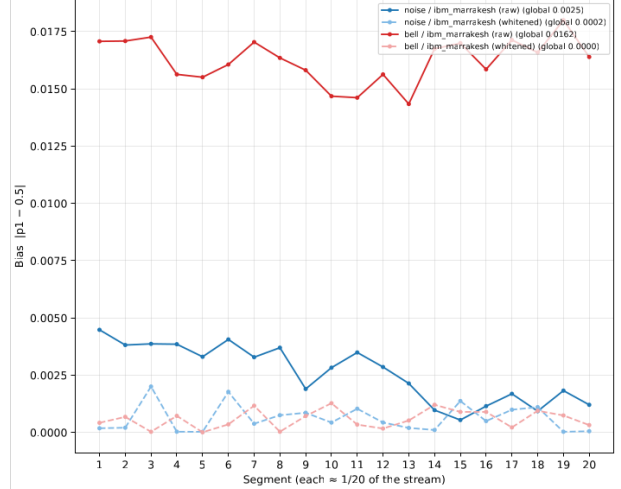


Fig. 3: Segment-wise bias $|p_1 - 0.5|$ across 20 equal partitions of each stream, IBM Marrakesh. Solid lines are raw streams, dashed lines are whitened. The ordering matches IBM Fez, indicating the effect is not processor-specific.

Table III reports the companion metric, per-bit min-entropy, which tracks the same ordering: raw Hadamard streams sit above 0.99 on both processors, while raw Bell-pair streams sit approximately 4–5 percentage points lower. After SHA-512 whitening, min-entropy converges to ≥ 0.9992 for every stream but whitening also discards 25% of the raw bits to reach that point, so the raw-stream gap still determines how many usable bits a given amount of QPU time buys (Sec. V-F).

TABLE III: Per-bit min-entropy by stream. Higher is better; ideal is 1.0.

Stream	Bits	Min-entropy/bit
Hadamard/Fez raw	10,540,000	0.9949
Hadamard/Fez white	7,905,000	0.9992
Bell/Fez raw	10,016,243	0.9504
Bell/Fez white	7,512,184	0.9994
Hadamard/Marr. raw	9,040,000	0.9927
Hadamard/Marr. white	6,780,000	0.9993
Bell/Marr. raw	9,798,011	0.9539
Bell/Marr. white	7,348,512	0.9999

C. NIST SP 800-22 Results

Table IV and Table V report, per processor, the number of 1,000,000-bit partitions (out of 5) that passed each NIST SP 800-22 test, with the overall pass rate on the bottom row. On raw streams, the Hadamard design achieves 0.62 (Fez) and 0.68 (Marrakesh); the Bell-pair design achieves 0.45 (Fez) and 0.48 (Marrakesh). The raw Bell-pair streams fail every bias-sensitive test (Monobit, Frequency Within Block, Cumulative Sums, Runs, Serial, Approximate Entropy) on all five partitions of both processors, while the raw Hadamard streams fail only a strict subset of those same tests. After

SHA-512 whitening, every whitened stream passes essentially all tests (overall pass rates 0.99–1.00).

TABLE IV: NIST SP 800-22 on IBM Fez: partitions passed out of 5 (raw = unwhitened, wh = whitened).

Test	Had raw	Had wh	Bell raw	Bell wh
Approx. Entropy	1/5	5/5	0/5	5/5
Binary Matrix Rank	5/5	5/5	5/5	5/5
Cumulative Sums	0/5	5/5	0/5	5/5
Discrete Fourier Transf.	5/5	4/5	5/5	5/5
Frequency Within Block	4/5	5/5	0/5	5/5
Linear Complexity	5/5	5/5	5/5	5/5
Longest Run of Ones	5/5	5/5	2/5	5/5
Maurer’s Universal	5/5	5/5	5/5	5/5
Monobit	0/5	5/5	0/5	5/5
Non-Overlapping Templ.	5/5	5/5	5/5	5/5
Random Excursion	–	4/4	–	4/4
Random Excursion Var.	–	4/4	–	4/4
Runs	1/5	5/5	0/5	5/5
Serial	1/5	5/5	0/5	5/5
OVERALL pass rate	0.62	0.99	0.45	1.00

TABLE V: NIST SP 800-22 on IBM Marrakesh: partitions passed out of 5 (raw = unwhitened, wh = whitened).

Test	Had raw	Had wh	Bell raw	Bell wh
Approx. Entropy	2/5	5/5	0/5	5/5
Binary Matrix Rank	5/5	5/5	5/5	5/5
Cumulative Sums	0/5	5/5	0/5	5/5
Discrete Fourier Transf.	5/5	5/5	5/5	5/5
Frequency Within Block	3/5	5/5	0/5	5/5
Linear Complexity	5/5	5/5	5/5	5/5
Longest Run of Ones	5/5	5/5	4/5	5/5
Maurer’s Universal	5/5	5/5	5/5	5/5
Monobit	2/5	5/5	0/5	5/5
Non-Overlapping Templ.	5/5	5/5	5/5	5/5
Random Excursion	–	3/3	–	5/5
Random Excursion Var.	–	3/3	–	5/5
Runs	2/5	5/5	0/5	5/5
Serial	2/5	5/5	0/5	5/5
OVERALL pass rate	0.68	1.00	0.48	1.00

D. Next-Bit Prediction

Table ?? reports next-bit prediction with logistic regression on a representative window (16 preceding bits). Because the primary signal is ROC-AUC with a Hanley–McNeil 95% CI, the metric is unaffected by bias itself: a stream that is heavily biased but structurally memoryless still records $AUC \approx 0.5$. All four raw streams, and their whitened versions, pass the strict test on every window there is no learnable short-range structure in either design on either processor. Note that the Bell-pair raw *accuracy* (0.516 on Fez, 0.517 on Marrakesh) is elevated relative to the Hadamard raw accuracy (0.500 on Fez, 0.503 on Marrakesh), but so is the majority baseline (0.517, 0.517 for Bell; 0.500, 0.505 for Hadamard). The accuracy elevation is entirely attributable to bias, not to learnable dependency, which is precisely what a bias-robust AUC metric is designed to reveal.

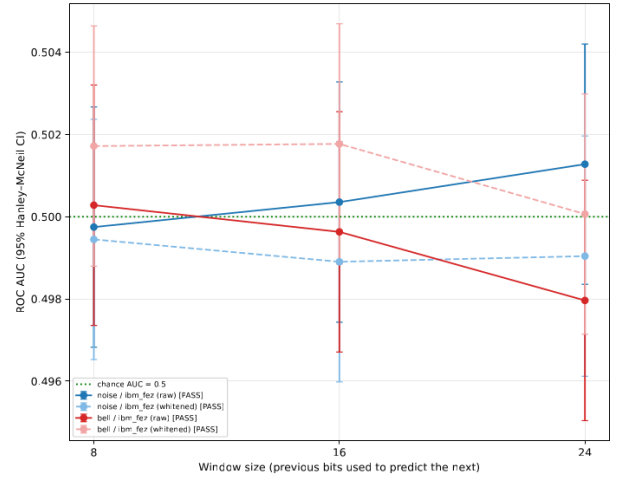


Fig. 4: next-bit prediction on IBM Fez: ROC-AUC vs. window size $W \in \{8, 16, 24\}$ with 95% Hanley–McNeil confidence intervals. Every stream’s CI includes the chance line at 0.5.

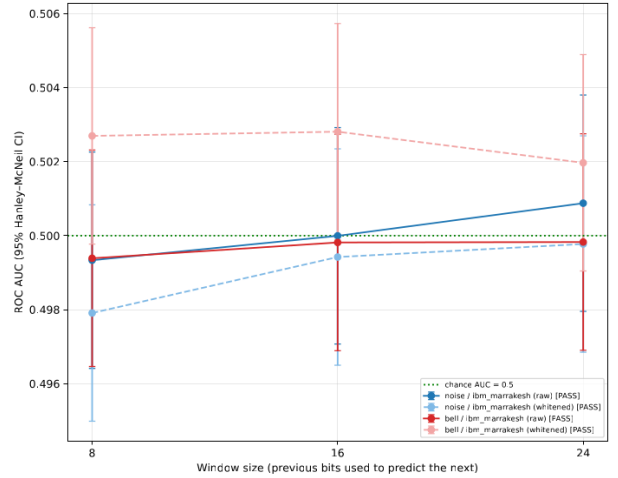


Fig. 5: next-bit prediction on IBM Marrakesh: ROC-AUC vs. window size $W \in \{8, 16, 24\}$ with 95% Hanley–McNeil confidence intervals. The raw accuracy elevation of the Bell-pair streams (Table ??) is entirely attributable to bias, not to learnable structure.

E. Multi-Order Markov Dependency

Table VII reports Markov predictor accuracy in excess of the majority baseline at orders 1, 2, and 4. The excess magnitudes are ≤ 0.0013 across all streams and orders, and the verdicts are uniformly PASS. Again, the raw bias itself does not lift these bars: this metric detects dependency, not bias.

TABLE VII: Markov dependency: accuracy above majority baseline (higher = more dependency). All streams pass.

Stream	o1 excess	o2 excess	o4 excess
Had./Fez raw	0.0012	0.0012	0.0009
Had./Fez white	0.0013	0.0005	-0.0019
Bell/Fez raw	0.0006	0.0006	0.0006
Bell/Fez white	0.0012	-0.0009	-0.0013
Had./Marr. raw	-0.0010	-0.0010	-0.0010
Had./Marr. white	-0.0028	-0.0023	-0.0005
Bell/Marr. raw	0.0013	0.0013	0.0013
Bell/Marr. white	-0.0015	-0.0015	-0.0010

F. Cost Comparison

Table VIII reports cost per 10^6 usable raw bits, computed from the empirical shot yield of each stream (i.e. after forbidden-state discard). On both processors, the Bell-pair design requires roughly $2.6\text{--}3.0\times$ more shots and $2.6\text{--}3.0\times$ more QPU-seconds per usable megabit than the Hadamard baseline; the USD cost tracks the QPU-seconds one-for-one.

TABLE VIII: Cost per 10^6 usable raw bits (empirical, after forbidden-state discard).

Design/Backend	Shots	Bits/shot	QPU s	Cost (\$)
Hadamard/Fez	8,065	124.0	2.47	3.95
Bell/Fez	23,961	41.7	7.29	11.66
Hadamard/Marrakesh	8,850	113.0	2.88	4.60
Bell/Marrakesh	23,474	42.6	7.55	12.08
Bell / Hadamard (F)		2.95$\times$		
Bell / Hadamard (M)		2.63$\times$		

G. Cost–Quality Summary

Combining the raw NIST pass rate (Tables IV and V, bottom rows) with the cost per usable megabit (Table VIII) yields the summary in Table IX. The Hadamard configurations dominate both axes simultaneously: higher quality and lower cost on both processors. No trade-off exists in the classical sense the Bell-pair design is Pareto-dominated.

TABLE IX: Cost–quality summary: raw NIST pass rate vs. USD per usable megabit.

Design/Backend	Raw NIST pass rate	USD / 10^6 bits
Hadamard/F	0.62	3.95
Bell/F	0.45	11.66
Hadamard/M	0.68	4.60
Bell/M	0.48	12.08

VI. DISCUSSION

A. Answering the Research Question

The research question was whether Bell-pair error detection produces better bits than a raw depth-1 Hadamard QRNG, and whether any advantage is worth its cost. The answer is **no on both counts**, at current IBM Heron fidelities: Bell-pair raw streams are $\sim 10\times$ more biased, have $\sim 4\text{--}5$ points lower min-entropy, fail more NIST tests, and cost $\sim 2.8\times$ more per usable

bit. The direction is identical on both processors, so this is not an artefact of one backend.

B. Why Bell-Pair Error Detection Failed to Help

Detecting forbidden states is correct in principle, but two mechanisms undermine it in practice:

Two-qubit gate error dominates. A Hadamard circuit exposes each bit to one single-qubit rotation and one measurement; a Bell-pair circuit adds a two-qubit gate (fidelity $\sim 99\%$ vs. $\sim 99.9\%$ for single-qubit gates on Heron) and a second measurement. Discarding forbidden states only catches errors that flip *one* qubit of the pair — correlated errors flipping both pass through undetected as biased $|00\rangle/|11\rangle$ outcomes. Residual bias after discard remains ~ 0.017 , confirming the detector catches only a small share of the effective bias.

Hadamard has fewer bias sources. Its only significant bias source is single-qubit readout asymmetry, averaging to ~ 0.002 across 100+ healthy qubits. Bell-pair adds entangling-gate infidelity and cross-talk, neither corrected by the discard filter.

Throughput. Bell-pair yields one bit per two qubits per shot, versus one bit per qubit for Hadamard — a $2\times$ disadvantage before quality is even considered.

C. Relationship to the Preliminary Study

The preliminary finding — include beats discard within the Bell-pair design — is not overturned here; it remains a valid policy-level observation. But the present work asks whether Bell-pair (either policy) beats a simpler alternative, and the answer is no: even taking the include-policy benefit at face value, bias would fall only to ~ 0.010 , still an order of magnitude above Hadamard and still $2.8\times$ more costly.

D. On the Role of Post-Processing

After SHA-512 whitening, all four streams pass every test identically — so does the raw comparison matter? Yes, for three reasons. Whitening discards 25% of raw bits, so any raw disadvantage becomes a proportional throughput penalty. Whitening cannot create entropy that isn’t there: Hadamard’s raw min-entropy (~ 0.995) versus Bell-pair’s (~ 0.951) reflects a real structural gap in the physical source, even when whitened bias looks identical [6]. And cost is incurred in QPU-seconds producing raw bits, not in the free offline whitening step — so a source needing more shots per usable bit is simply more expensive.

E. Cost–Quality Trade-off: When Would Bell Ever Win?

Two changes could make Bell-pair worthwhile: (i) entangling-gate fidelity improving by roughly an order of magnitude relative to readout error, closing the quality gap, though cost parity would additionally require topology-bounded yield improvements; or (ii) a fault-tolerant regime with active error correction on the entangling operation, the follow-up direction anticipated in [9]. Until then, a fault-tolerant Bell-pair QRNG would need to beat Hadamard on quality *and* justify its error-correction overhead.

F. Scientific Value of a Negative Result

This is a negative result in the technical sense: an intuitive hypothesis — error detection improves QRNG quality — is refuted by controlled measurement. Such results matter: they prevent unnecessary complexity and redirect effort toward mechanisms with a realistic chance of working. Here, the cheapest design (depth-1 Hadamard) is also the best-quality design on current IBM Heron hardware; Bell-pair QRNGs merit revisiting only once entangling-gate fidelity improves substantially or fault-tolerant execution becomes practical.

G. Limitations

Three limitations bound this conclusion. First, both processors are IBM Heron rev 2; the result may not generalise to other architectures (Google Willow, IQM Emerald, Rigetti Ankaa), ion-trap systems (IonQ, Quantinuum), or photonic platforms. Second, we implemented error *detection*, not true fault-tolerant QEC, which was infeasible on Heron-class hardware without dedicated ancilla infrastructure. Third, the experiment spans a bounded calibration window; IBM’s calibration can shift across weeks, so exact bias values should not be treated as processor invariants.

H. Future Work

Worthwhile extensions include: replication on non-IBM superconducting hardware (IQM Emerald, Rigetti Ankaa) and on ion-trap/photonic platforms, to test generality; repeating the comparison once fault-tolerant execution is routinely available, since the Hadamard advantage may erode there; and a broader sweep across other bit-generating circuit families (cluster-state, high-dimensional entangled QRNGs) under a common cost–quality metric.

VII. CONCLUSION

We asked whether the error detection built into a Bell-pair QRNG produces statistically better bits than a raw, depth-1 Hadamard QRNG on current superconducting hardware, and whether any quality advantage justifies the design’s cost premium. Across two IBM Heron processors and roughly 4×10^7 raw bits, the answer is unambiguously negative on both counts.

The Hadamard baseline delivered an order-of-magnitude lower raw bias (0.0018 and 0.0025 vs. 0.0175 and 0.0162), higher raw min-entropy (0.9949 and 0.9927 vs. 0.9504 and 0.9539), higher raw NIST SP 800-22 pass rates (0.62 and 0.68 vs. 0.45 and 0.48), and approximately $2.8\times$ lower cost per usable megabit. Both raw and whitened streams passed the strict next-bit and Markov-dependency tests, so neither design contains detectable short-range structure the difference is entirely a difference in bias, and bias is the axis on which the Bell-pair design loses.

The preliminary study’s earlier finding about forbidden-state policy is preserved as an internal claim about Bell-pair QRNGs but does not rescue the design at the cost–quality frontier: even a favourable policy would leave the Bell-pair bias an order of magnitude above the Hadamard baseline while retaining the $2.8\times$ cost premium. On current IBM Heron hardware, the

added circuit complexity of Bell-pair QRNGs is therefore not justified, and the simple Hadamard design is objectively the better choice. Whether that ordering survives into the fault-tolerant regime is the next question worth asking.

ACKNOWLEDGMENT

The author thanks IBM Quantum for access to the Fez and Marrakesh processors through the Open Plan, and the Department of Computers and Informatics of the Technical University of Košice for computational support.

REFERENCES

- [1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th ed. Cambridge University Press, 2010.
- [2] M. Herrero-Collantes and J. C. Garcia-Escartin, “Quantum random number generators,” *Rev. Mod. Phys.*, vol. 89, no. 1, p. 015004, 2017.
- [3] L. E. Bassham et al., “A statistical test suite for random and pseudorandom number generators for cryptographic applications,” NIST Special Publication 800-22 Rev. 1a, 2010.
- [4] M. Blum and S. Micali, “How to generate cryptographically strong sequences of pseudorandom bits,” *SIAM J. Comput.*, vol. 13, no. 4, pp. 850–864, 1984.
- [5] L. K. Shalm et al., “Strong loophole-free test of local realism,” *Phys. Rev. Lett.*, vol. 115, no. 25, p. 250402, 2015.
- [6] B. Barak, R. Shaltiel, and E. Tromer, “True random number generators secure in a changing environment,” in *Cryptographic Hardware and Embedded Systems – CHES 2003*, LNCS 2779, Springer, 2003, pp. 166–180.
- [7] IBM Quantum, “Qiskit Runtime primitives,” <https://quantum.cloud.ibm.com/docs/guides/qiskit-runtime-primitives>, accessed 2026.
- [8] IBM Quantum, “The Heron processor family,” IBM Research Blog, 2024.
- [9] P. Jaš, “Bell pair QRNG: design, implementation, and the effect of forbidden state inclusion on bit-stream bias,” Technical University of Košice, 2026.
- [10] G. Marsaglia, “The Marsaglia random number CDROM including the Diehard battery of tests of randomness,” 1995.
- [11] P. L’Ecuyer and R. Simard, “TestU01: A C library for empirical testing of random number generators,” *ACM Trans. Math. Softw.*, vol. 33, no. 4, art. 22, 2007.
- [12] A. Acín and L. Masanes, “Certified randomness in quantum physics,” *Nature*, vol. 540, no. 7632, pp. 213–219, 2016.
- [13] S. Pironio et al., “Random numbers certified by Bell’s theorem,” *Nature*, vol. 464, no. 7291, pp. 1021–1024, 2010.
- [14] P. Krantz et al., “A quantum engineer’s guide to superconducting qubits,” *Appl. Phys. Rev.*, vol. 6, no. 2, p. 021318, 2019.